

Análise de Interações no Repositório `giscus/giscus` utilizando Teoria de Grafos

Isabella Luiza Dias dos Santos¹
Islayder Jackson Ribeiro de Oliveira²
Luiz Arthur Costa e Costa³
Sofia Vasconcelos Moreira e Silva⁴

¹Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-000 – Belo Horizonte – MG – Brasil

{ildsantos, islayder, luiz.arthur, sofia.silva.1525938}@sga.pucminas.br

Abstract. *This paper presents a Java 21 tool for mining and analyzing contributor interactions in the `giscus/giscus` repository using custom directed graphs (adjacency lists and matrices, no external graph libraries). It builds four networks and computes eleven topological metrics, with CLI and Desktop interfaces, GEXF export for Gephi, and validation through 167 unit tests and the `GraphApiDemo` application.*

Resumo. *Este artigo apresenta uma ferramenta em Java 21 para minerar e analisar interações no repositório `giscus/giscus` por meio de grafos direcionados implementados nativamente (sem bibliotecas externas). São construídos quatro grafos, calculadas onze métricas topológicas, oferecidas interfaces CLI e Desktop, exportação GEXF para o Gephi e validação com 167 testes unitários e a aplicação `GraphApiDemo`.*

1. Introdução

O desenvolvimento de software de código aberto (OSS) caracteriza-se por redes complexas de colaboração. No GitHub, interações como discussões em issues, revisões de código, aprovações e *merges* moldam a evolução e a governança dos projetos. Este trabalho propõe uma ferramenta que extrai, processa e analisa essas interações utilizando teoria de grafos: colaboradores são vértices e ações direcionadas são arestas (com pesos no grafo integrado).

O repositório alvo é `giscus/giscus`, sistema de comentários baseado em GitHub Discussions, com mais de 5.000 estrelas, comunidade ativa e volume adequado de issues e pull requests. Em contraste com repositórios massivos como `facebook/react`, o `giscus` permite mineração integral e demonstração ao vivo respeiando tempo de processamento e *rate limits* da API.

O plano de desenvolvimento seguiu três etapas do enunciado: (1) modelagem e mineração; (2) API de grafos, interfaces e testes; (3) métricas topológicas, exportação GEXF e relatório. Entregas intermediárias foram versionadas no repositório Git do grupo.

Tabela 1. Autores e Funções desempenhadas no projeto.

Autor	Função
Isabella Luiza Dias dos Santos	Mineração de dados, construção dos grafos, elaboração do relatório
Islayder Jackson Ribeiro de Oliveira	Desenvolvimento da aplicação core
Luiz Arthur Costa e Costa	Desenvolvimento da aplicação core
Sofia Vasconcelos Moreira e Silva	Mineração de dados, construção dos grafos, elaboração do relatório

Este artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve arquitetura, domínio, coleta e API de grafos; a Seção 4 lista as métricas implementadas; a Seção 5 apresenta resultados visuais e numéricos; e a Seção 6 conclui o trabalho.

2. Revisão de Literatura

A teoria de grafos fornece base matemática para Análise de Redes Sociais (SNA). No ecossistema GitHub, o *social coding* transformou repositórios em redes de comunicação e revisão por pares [Dabbish et al. 2012]. Representar usuários como vértices e ações como arestas direcionadas permite identificar fluxos de informação e papéis estruturais.

Métricas locais, como o grau, destacam participação direta. A centralidade de intermediação (*betweenness*) identifica vértices que atuam como pontes entre subcomunidades [Freeman 1977]. Algoritmos iterativos como PageRank [Page et al. 1999] e centralidade de autovetor ponderam não só a quantidade de conexões, mas a centralidade estrutural dos vizinhos conectados. Métricas de estrutura (densidade, aglomeração, assortatividade) e de comunidade (modularidade [Newman 2006], *bridging ties*) complementam a análise de ecossistemas OSS.

3. Arquitetura e Modelagem do Problema

A solução foi desenvolvida em Java 21 com Maven, dividida em camadas: `app` (CLI e desktop), `github` (coleta), `model` (domínio), `service` (construção dos grafos), `graph` (API própria), `analysis` (métricas) e `export` (GEXF). Jackson é usado apenas para interpretar JSON da API, e não como biblioteca de grafos.

3.1. Arquitetura em Camadas

A Figura 1 resume o fluxo principal: o usuário aciona CLI ou desktop; a coleta produz `RepositoryData`; o `GraphBuilderService` gera quatro grafos; serviços de análise e exportação produzem artefatos em `output/` e arquivos GEXF para o Gephi.

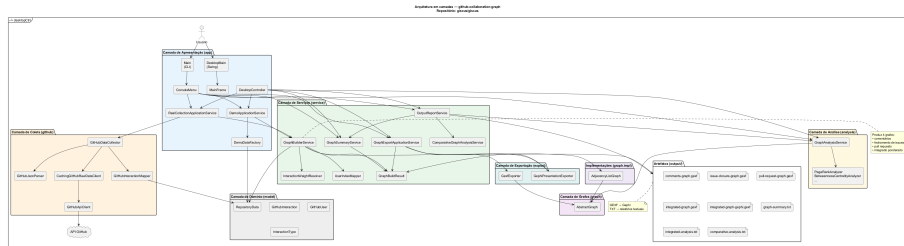


Figura 1. Arquitetura em Camadas e Fluxo de Dados do Sistema.

[Link PNG Arquitetura em Camadas](#)

3.2. Modelagem de Domínio

O pacote `model` representa o problema independentemente da API de grafos da disciplina (Figura 2):

- `GitHubUser`: colaborador identificado pelo `login`;
- `GitHubInteraction`: relação direcionada `sourceUser` → `targetUser` com `InteractionType`;
- `RepositoryData`: agregado com `owner`, `repository`, lista de interações e mapa de PRs abertos por autor;
- `InteractionWeights`: constantes de peso (2, 3, 4, 5) alinhadas ao enunciado.

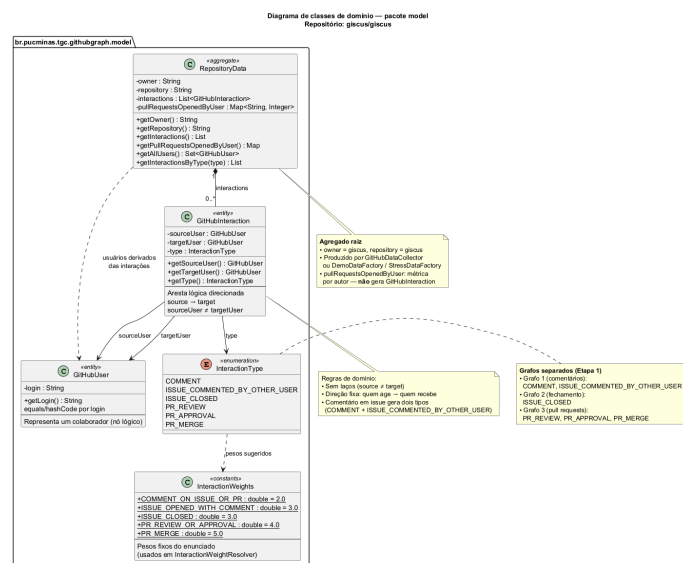


Figura 2. Diagrama de Classes do Domínio.

[Link PNG Diagrama de Classes do Domínio](#)

3.3. Estratégia de Coleta e Mapeamento para Arestas

A coleta opera em modo `FULL_REPOSITORY`: paginação completa de issues, pull requests, comentários e revisões, com `concurrency` configurável. A coleta possui suporte a cache local em `cache/github/`. Na execução de referência, os dados foram obtidos por mineração completa do repositório, sem reutilização prévia de respostas armazenadas em cache. Nessa execução foram processados 377 issues, 1054 pull requests,

4294 interações e 566 colaboradores distintos, em tempo de parede de aproximadamente 2min51s.

Tabela 2. Transformação de eventos GitHub em interações e grafos.

Evento GitHub	Origem → Destino	Grafo	Observação
Comentário em issue/PR	autor do comentário → autor do item	Comentários	Peso 2 no integrado
Comentário de terceiro em issue	autor → autor da issue	Comentários	Gera também tipo com peso 3
Fechamento de issue	quem fechou → autor da issue	Fechamento	Peso 3 no integrado
Review de PR	revisor → autor do PR	Pull requests	Peso 4
Aprovação de PR	revisor → autor do PR	Pull requests	Peso 4
Merge de PR	quem mergeou → autor do PR	Pull requests	Peso 5
Abertura de PR	–	Nenhum	Contagem por autor (Tabela 7)

3.4. Construção dos Quatro Grafos e Pesos

O `GraphBuilderService` mapeia logins para índices inteiros via `UserIndexMapper` (ordem alfabética) e instancia quatro `AdjacencyListGraph`. Nos grafos **separados**, há no máximo uma aresta por par ordenado (u, v) , com peso fixo do tipo na primeira ocorrência. No grafo **integrado**, os pesos são **acumulados** quando o par já existe. O grafo é direcionado e simples: sem laços, sem arestas duplicadas no mesmo sentido; arestas antiparalelas $(u \rightarrow v)$ e $(v \rightarrow u)$ são permitidas.

Tabela 3. Pesos fixos atribuídos por tipo de interação no grafo integrado.

Tipo de Interação	Peso
Comentário em issue ou pull request	2
Comentário de terceiro em issue	3
Fechamento de issue	3
Revisão ou aprovação de pull request	4
Merge de pull request	5

Tabela 4. Contagem de vértices/arestas (mineração de referência).

Grafo	V	E
Comentários	566	698
Fechamento de issues	566	155
Pull requests	566	51
Integrado ponderado	566	737

3.5. API Própria de Grafos

Conforme exigido na Etapa 2, a estrutura topológica parte de `AbstractGraph`, com implementações `AdjacencyMatrixGraph` e `AdjacencyListGraph`. A API inclui contagem de vértices e arestas, consulta e mutação de arestas, relações estruturais (`isSucessor`, `isPredessor`, `isDivergent`, `isConvergent`, `isIncident`), graus, pesos de vértices e arestas, `isConnected`, `isEmptyGraph`, `isCompleteGraph` e `exportToGEPHI`. A operação `addEdge` é idempotente; índices inválidos e laços lançam exceções.

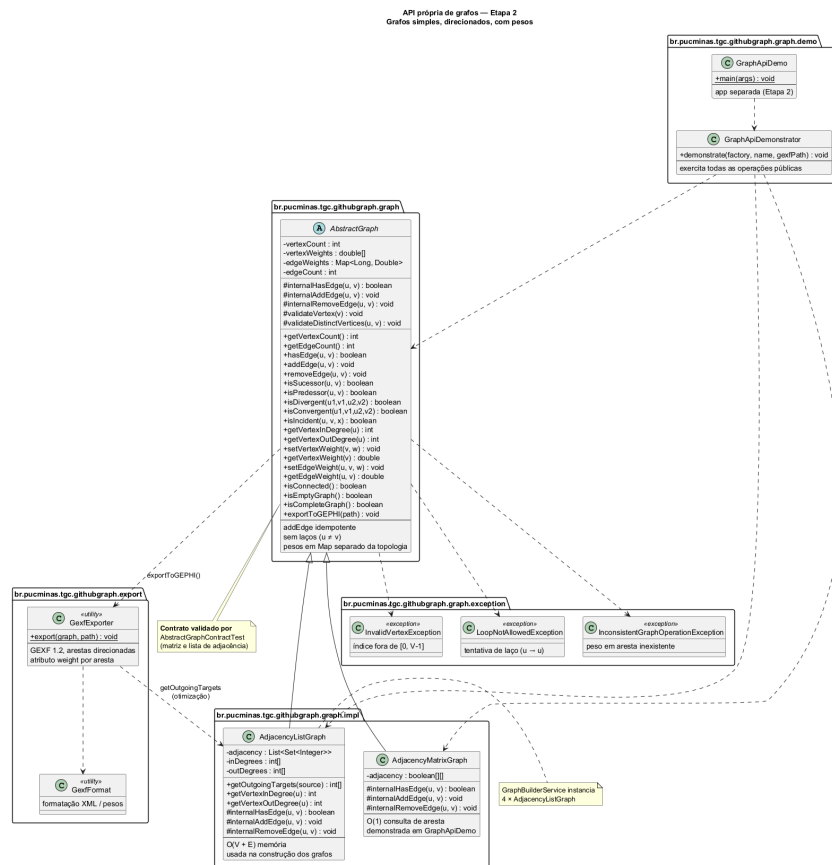


Figura 3. Diagrama de Classes da API de Grafos

[Link PNG Diagrama de Classes da API](#)

`AdjacencyMatrixGraph` favorece verificação $O(1)$ de arestas; `AdjacencyListGraph` otimiza memória em $O(V + E)$ e é a representação usada na construção dos grafos de colaboração.

3.6. Interfaces e Demonstração

Três pontos de entrada consomem a mesma camada de serviços:

1. **CLI (Main):** menu interativo para mineração real, demonstração offline (contingência), resumo dos grafos, análise de métricas e exportação GEXF;
2. **Desktop Swing (DesktopMain):** interface gráfica com *SwingWorkers*, barra de progresso e logs da mineração em segundo plano;

3. **GraphApiDemo:** aplicação **separada** que instancia matriz e lista de adjacência e demonstra **todas** as operações públicas de AbstractGraph, incluindo validações e exportToGEPHI.

```
== github-collaboration-graph ==
Repositorio configurado: giscus/giscus (mineracao completa FULL_REPOSITORY)
1. Mostrar informacoes do projeto
2. Minerar repositorio inteiro configurado (fluxo principal)
3. Executar demonstracao offline (plano B)
4. Exibir resumo dos grafos carregados
5. Analisar grafo integrado
6. Exportar todos os grafos para GEXF
7. Gerar relatorios em output/
8. Sair
9. Modo stress offline (sem API: mede limite interno)
Escolha uma opcao: 4

Mineracao concluida.
Tempo total: 175,98 s (coleta: 175,96 s, grafos: 0,01 s)
Requisicoes API: 2443 | Acertos de cache: 0 (pasta cache/github/)
Usuarios (vertices): 568
Interacoes coletadas: 4302
Arestas no grafo integrado: 739
Issues listadas: 377
Pull requests listados: 1058
Comentarios de issues: 303 chamadas, 74 puladas por comments=0
Comentarios de PRs: 1058 chamadas, 0 puladas por comments=0
Reviews de PRs: 1058 chamadas
Chamadas totais estimadas evitadas: 74
Tempo listagem issues: 16s
Tempo listagem PRs: 16s
Tempo comentarios issues: 1min35s
Tempo comentarios PRs: 5min53s
Tempo reviews: 6min43s
Tempo construcao dos grafos: 11ms
Tempo total: 2min55s

Resumo da analise de colaboracao
Usuarios (vertices): 568
Usuarios mapeados: 01joy, 10Nome-traducos, iKevinson, 21km43, 2Draining, 596094101, 5ideceal, aayushdutt, AbdeitwabMF, ABeltramo, ABGEO, AbhiVarde, abjutus

Grafo de comentarios
Vertices: 568
Arestas: 700
Vazio: false
Completo: false

Grafo de fechamento de issues
Vertices: 568
Arestas: 155
Vazio: false
Completo: false

...
```

Figura 4. Interface CLI (Menu interativo no terminal) para execução da mineração.

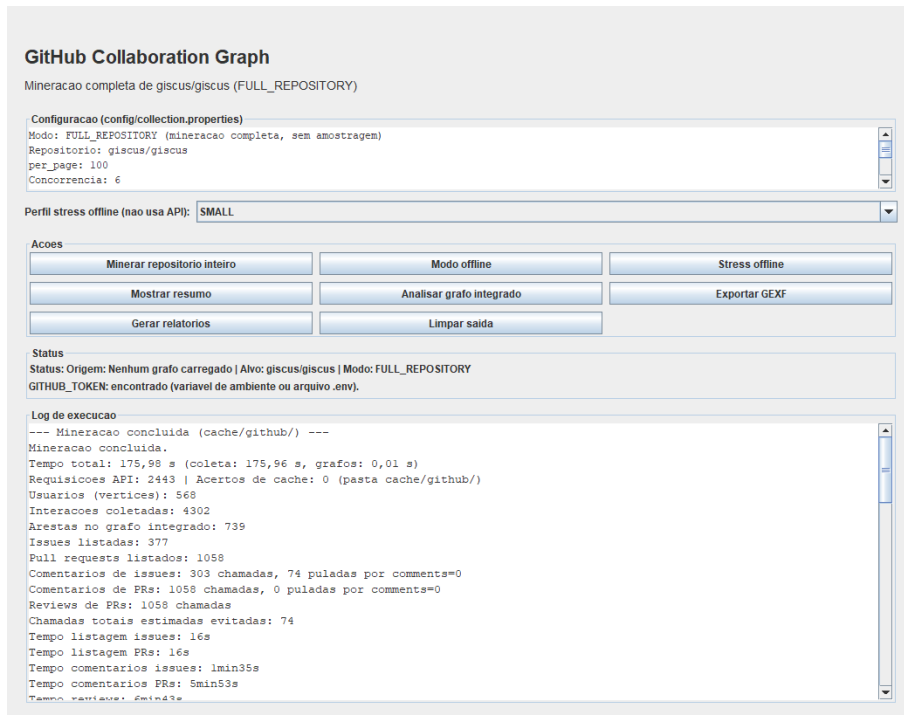


Figura 5. Interface Gráfica Desktop desenvolvida em Java Swing exibindo logs de extração.

3.7. Validação e Testes

O projeto possui **167 testes unitários** (JUnit 5), executados offline. Destacam-se: `AbstractGraphContractTest` (contrato da API para matriz e lista); `GitHubInteractionMapperTest` e `GitHubDataCollectorTest` (mineração e mapeamento); `GraphBuilderServiceTest` (quatro grafos e pesos acumulados); testes dos analisadores (PageRank, Brandes, comunidades, *bridging ties*); `GexfExporterTest` e `GraphApiDemonstratorTest`. Nenhum teste depende da API real do GitHub.

4. Métricas e Algoritmos Implementados

Todas as métricas são calculadas manualmente sobre o grafo integrado, sem JGraphT, NetworkX ou equivalentes. O `GraphAnalysisService` orquestra onze blocos de análise, materializados no arquivo `integrated-analysis.txt`:

1. **Grau (Degree):** graus de entrada, saída e total;
2. **Densidade:** $E/(V(V - 1))$ para grafo direcionado simples;
3. **PageRank:** iteração com *damping* 0,85; em grafo direcionado, mede centralidade recebida na rede;
4. **Centralidade de autovetor:** *power iteration* nas arestas de entrada; indica relevância como destino das interações;
5. **Closeness:** BFS direcional para caminhos mínimos não ponderados, com tratamento específico para vértices inalcançáveis em grafos desconexos;
6. **Betweenness (Brandes):** caminhos mínimos em grafo direcionado não ponderado;
7. **Clustering:** coeficiente local médio em vizinhança fraca;
8. **Assortatividade:** correlação de Pearson entre graus totais fracos dos extremos das arestas;
9. **Comunidades:** *label propagation* determinístico + modularidade aproximada;
10. **Bridging ties (laços-ponte):** arestas entre comunidades distintas ranqueadas por score;
11. **Aberturas de PR:** contagem por autor (sem aresta no grafo).

Em grafos com mais de 8.000 vértices, métricas pesadas (eigenvector, closeness, betweenness, comunidades) podem ser omitidas automaticamente; no `giscus/giscus` (566 vértices), todas foram calculadas.

5. Resultados e Análise

A mineração integral produziu a rede da Tabela 5. A densidade global foi de **0,0023**, indicando rede esparsa típica de OSS. O coeficiente médio de aglomeração foi **0,1949** e a assortatividade $-0,2767$, sugerindo que colaboradores de graus diferentes tendem a interagir entre si.

5.1. Análise Visual dos Grafos

As Figuras 6–9 foram geradas a partir dos arquivos GEXF exportados pela ferramenta; o grafo integrado reutiliza o layout do `integrated-graph-gephi.gexf` (Gephi), com *logins* reais. O grafo de comentários é o mais denso (698 arestas); o de fechamento

concentra mantenedores (155); o de PRs isola revisão e merge (51); o integrado consolida pesos acumulados (737).

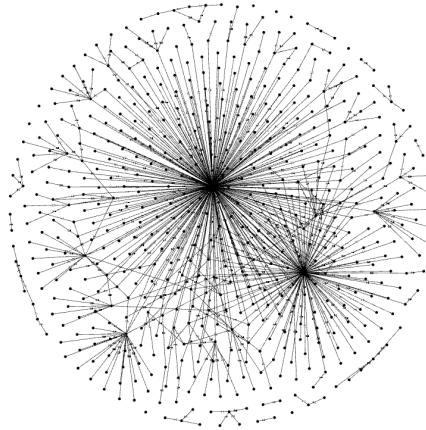


Figura 6. Visualização da sub-rede correspondente ao Grafo de Comentários.

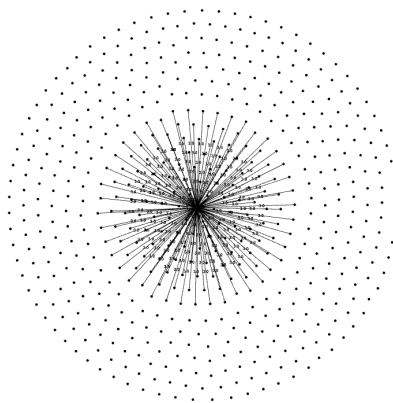


Figura 7. Visualização do Grafo de Fechamento de Issues.

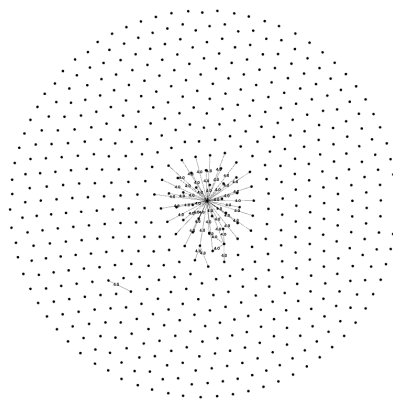


Figura 8. Visualização do Grafo focado em Revisões e Merges de Pull Requests.

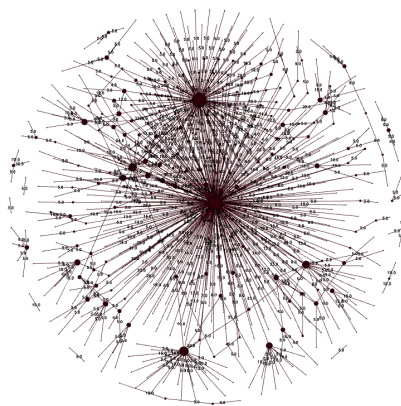


Figura 9. Visualização do Grafo Integrado Ponderado completo.

5.2. Métricas Globais e de Estrutura

Tabela 5. Métricas globais e estruturais do grafo integrado.

Métrica	Valor
Vértices	566
Arestas	737
Densidade	0,0023
Clustering (média)	0,1949
Assortatividade (Pearson)	-0,2767
Comunidades detectadas	37
Maior comunidade	370 vértices (65,37%)
Modularidade (aprox.)	0,3563
PRs abertos (total)	1054

A detecção de comunidades identificou um núcleo dominante (comunidade 265, centrada em *laymonage*, com 370 membros incluindo *vercel[bot]* e *nschonni*) e comunidades menores (ex.: comunidade 90 com *CheeseB*, 31 membros).

5.3. Centralidades Estruturais

A Tabela 6 apresenta os cinco primeiros colaboradores em métricas de centralidade. Como cada aresta segue a direção autor da ação → usuário associado ao artefato afetado, PageRank e centralidade de autovetor devem ser lidos como centralidade recebida ou relevância como destino das interações, e não como prova de influência direta. O usuário *laymonage* lidera grau total (338), betweenness (15262,33) e PageRank (0,0428), indicando posição estrutural central na rede de interações observada. *vercel[bot]* aparece em segundo no grau (131), reflexo de automação de *deploy*.

Tabela 6. Top 5 colaboradores por centralidade estrutural.

Grau total			Betweenness (Brandes)		
Usuário	Grau	Rank	Usuário	Score	Rank
laymonage	338	1	laymonage	15262,3333	1
vercel[bot]	131	2	ghost	604,0000	2
CheeseB	28	3	williamhorning	445,5000	3
nschonni	18	4	ouuan	303,5000	4
marcosruiz	16	5	sohang3112	303,0000	5

PageRank			Eigenvector		
Usuário	Score	Rank	Usuário	Score	Rank
laymonage	0,0428	1	laymonage	0,2099	1
CheeseB	0,0288	2	marcosruiz	0,1374	2
marcosruiz	0,0157	3	CheeseB	0,1104	3
antulik	0,0114	4	refparo	0,0970	4
ceynri	0,0095	5	Canfry	0,0835	5

Closeness: Closeness foi mantida por completude, mas em grafos direcionados esparsos seu ranking pode destacar vértices periféricos localizados em componentes pequenas. Por isso, seus resultados devem ser interpretados junto com grau, PageRank e betweenness, evitando conclusões isoladas sobre centralidade global.

5.4. Bridging Ties e Aberturas de PR

As bridging ties (laços-ponte) mais relevantes conectam a comunidade principal (265) a comunidades menores. Exemplos: *laymonage* → *CheeseB* (comunidades 265/90, peso 13,0); *laymonage* → *pourmand1376* (265/90, peso 16,0); *ghost* → *laymonage* (158/265, peso 10,0). Essas arestas indicam colaboradores que funcionam como pontes entre grupos que, de outra forma, interagiriam pouco.

Tabela 7. Top 5 – Aberturas de pull requests (métrica sem aresta).

Usuário	PRs abertos
dependabot[bot]	674
laymonage	213
nschonni	10
nidexingg	5
Andre601	3

A predominância de *dependabot[bot]* reflete automação de dependências; *laymonage* lidera entre contribuidores humanos na abertura de PRs.

6. Conclusão

A análise do repositório *giscus/giscus* demonstrou que a ferramenta desenvolvida compila dados heterogêneos de colaboração em redes direcionadas analisáveis. Os quatro grafos permitiram separar comunicação (698 arestas), fechamento e gerenciamento de issues (155), revisão técnica (51) e visão integrada ponderada (737).

Os resultados indicam comunicação amplamente distribuída em comentários, mas concentração de revisão, *merge* e fechamento em poucos mantenedores, notadamente *laymonage*, com posição estrutural destacada em grau, betweenness, PageRank e centralidade de autovetor. Métricas estruturais reforçam rede esparsa (densidade 0,0023), assortatividade negativa e comunidade principal com 65 % dos vértices.

A implementação nativa da API de grafos, validada por 167 testes e pela aplicação *GraphApiDemo*, atende aos requisitos das três etapas do trabalho. A exportação GEXF e as onze métricas topológicas transformam a ferramenta em instrumento de análise visual e quantitativa aplicável a outros ecossistemas open source de porte semelhante.

Referências

- [Dabbish et al. 2012] Dabbish, L., Stuart, C., Tsay, J., and Herbsleb, J. (2012). Social coding in github: Transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work*, pages 1277–1286.
- [Freeman 1977] Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1):35–41.
- [Newman 2006] Newman, M. E. J. (2006). Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23):8577–8582.
- [Page et al. 1999] Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The pagerank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab.